

1. Sliding Window Maximum

Problem: Given an array `nums` and a sliding window size `k`, find the maximum value in each window.

Test Cases

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`
Output: `[3,3,5,5,6,7]`

Input: `nums = [9, 11, 8, 5, 7, 10]`, `k = 2`
Output: `[11, 11, 8, 7, 10]`

Input: `nums = [4,2,1,3,5,6]`, `k = 4`
Output: `[4,5,6]`

Input: `nums = [1]`, `k = 1`
Output: `[1]`

Input: `nums = [10, 9, 8, 7, 6, 5, 4]`, `k = 3`
Output: `[10, 9, 8, 7, 6]`

2. Longest Substring with At Most K Distinct Characters

Problem: Find the length of the longest substring in a given string that contains at most k distinct characters.

Test Cases

Input: `s = "eceba"`, `k = 2`
Output: 3 (Substring: "ece")

Input: `s = "aa"`, `k = 1`
Output: 2 (Substring: "aa")

Input: `s = "aabbcc"`, `k = 3`
Output: 6 (Substring: "aabbcc")

Input: `s = "abcdef"`, `k = 2`
Output: 2 (Substring: "ab", "bc", etc.)

Input: `s = "aaaaaaa"`, `k = 1`
Output: 7 (Substring: "aaaaaaa")

3. Longest Palindromic Substring

Problem: Given a string `s`, find the longest substring that is a palindrome.

Test Cases

Input: `s = "babad"`
Output: "bab" or "aba"

Input: `s = "cbabd"`
Output: "bb"

Input: `s = "a"`

Output: "a"

Input: s = "abcd"

Output: "a" (or any single character)

Input: s = "racecar"

Output: "racecar"

4. Edit Distance

Problem: Compute the minimum number of operations (insertions, deletions, or substitutions) required to convert one string into another.

Test Cases

Input: word1 = "horse", word2 = "ros"

Output: 3

Input: word1 = "intention", word2 = "execution"

Output: 5

Input: word1 = "abc", word2 = "yabd"

Output: 2

Input: word1 = "kitten", word2 = "sitting"

Output: 3

Input: word1 = "", word2 = "abc"

Output: 3

5. Word Ladder II

Problem: Find all shortest transformation sequences from beginWord to endWord using words in the dictionary.

Test Cases

Input: beginWord = "hit", endWord = "cog", wordList = ["hot", "dot", "dog", "lot", "log", "cog"]

Output: [["hit", "hot", "dot", "dog", "cog"], ["hit", "hot", "lot", "log", "cog"]]

Input: beginWord = "hit", endWord = "cog", wordList = ["hot", "dot", "dog", "lot", "log"]

Output: []

Input: beginWord = "a", endWord = "c", wordList = ["a", "b", "c"]

Output: [["a", "c"]]

Input: beginWord = "abc", endWord = "xyz", wordList = ["abc", "xbc", "xyc", "xyz"]

Output: [["abc", "xyc", "xyz"]]

Input: beginWord = "red", endWord = "bed", wordList = ["red", "bed"]

Output: [["red", "bed"]]

6. N-Queens

Problem: Place n queens on an $n \times n$ chessboard such that no two queens threaten each other.

Test Cases

Input: n = 1

Output: [["Q"]]

Input: n = 2

Output: []

Input: n = 4

Output: [[".Q..", "...Q", "Q...", "..Q."], [".Q..", "Q...", "...Q", ".Q.."]]

Input: n = 8

Output: (Multiple valid solutions)

Input: n = 3

Output: []

7. Sudoku Solver

Problem: Write a program to solve a Sudoku puzzle by filling the empty cells.

Test Cases

Input: (Sudoku board with empty cells)

Output: (Solved Sudoku board)

Input: (Sudoku board where one number is missing)

Output: (Completed board with missing number filled)

Input: (Sudoku board with multiple empty cells)

Output: (Unique valid solution)

Input: (Board with an already solved Sudoku)

Output: (Same board)

Input: (Board with invalid placement)

Output: (No solution)

8. Palindrome Partitioning II

Problem: Given a string s, partition s such that every substring of the partition is a palindrome. Return the minimum cuts needed.

Test Cases

Input: s = "aab"

Output: 1 (["aa", "b"])

Input: s = "racecar"

Output: 0 (["racecar"])

Input: s = "abcd"

Output: 3 (["a", "b", "c", "d"])

Input: s = "civic"

Output: 0 (["civic"])

Input: s = "banana"

Output: 1 (["b", "anana"])

9. Binary Tree Maximum Path Sum

Problem: Given a non-empty binary tree, find the maximum path sum.

Test Cases

Input: [1,2,3]

Output: 6

Input: [-10,9,20,null,null,15,7]

Output: 42

Input: [2,-1]

Output: 2

Input: [-3]

Output: -3

Input: [5,4,8,11,null,13,4,7,2,null,null,null,1]

Output: 48

10. Serialize and Deserialize Binary Tree

Problem: Design an algorithm to serialize and deserialize a binary tree.

Test Cases

Input: [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5] (Serialized and deserialized correctly)

Input: []

Output: [] (Empty tree remains empty)

Input: [5,3,6,2,4,null,7]

Output: [5,3,6,2,4,null,7] (Preserves structure)

Input: [1]

Output: [1]

Input: [1,null,2,3]

Output: [1,null,2,3]